

Algorithm:

Step 1: Input & Initialization

- a. Read number of vertices V
- b. Read adjacency matrix $\text{graph}[V][V]$
- c. Create:
 - $\text{path}[V] \rightarrow$ stores the cycle
 - $\text{visited}[V] \rightarrow$ marks visited vertices

Step 2: Start Setup ($\text{findCycle}()$)

- a. Initialize:
 - $\text{path}[] = -1$
 - $\text{visited}[] = \text{false}$
- b. Start from vertex 0:
 - $\text{path}[0] = 0$
 - $\text{visited}[0] = \text{true}$
 - Call: $\text{solve}(1)$

Step 3: Recursive Function: $\text{solve}(\text{pos})$

- a. Base Case
 - if $\text{pos} == V$:
 - check if last vertex connects to first
 - if $\text{graph}[\text{path}[V-1]][\text{path}[0]] == 1$:
 - return true
 - else:
 - return false
- b. Try all vertices
 - for $v = 1$ to $V-1$:

Step 4: Safety Check ($\text{isSafe}(v, \text{pos})$)

- a. A vertex is valid if:
 - $\text{graph}[\text{path}[\text{pos}-1]][v] == 1$
 - $\text{visited}[v] == \text{false}$

Step 5: Place Vertex

$\text{path}[\text{pos}] = v$

$\text{visited}[v] = \text{true}$

Step 6: Recurse

- a. if $\text{solve}(\text{pos} + 1)$:
 return true

Step 7: Backtrack

- a. If not successful:
 $\text{visited}[v] = \text{false}$
 $\text{path}[\text{pos}] = -1$

Step 8: If All Fail return false

Step 9: Final Output

If solution found $\rightarrow$ print cycle

Else $\rightarrow$ "No Hamiltonian Cycle exists"